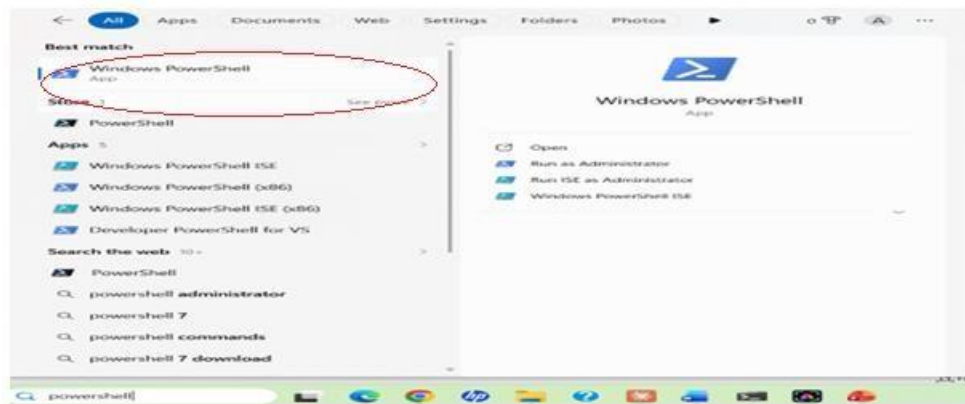
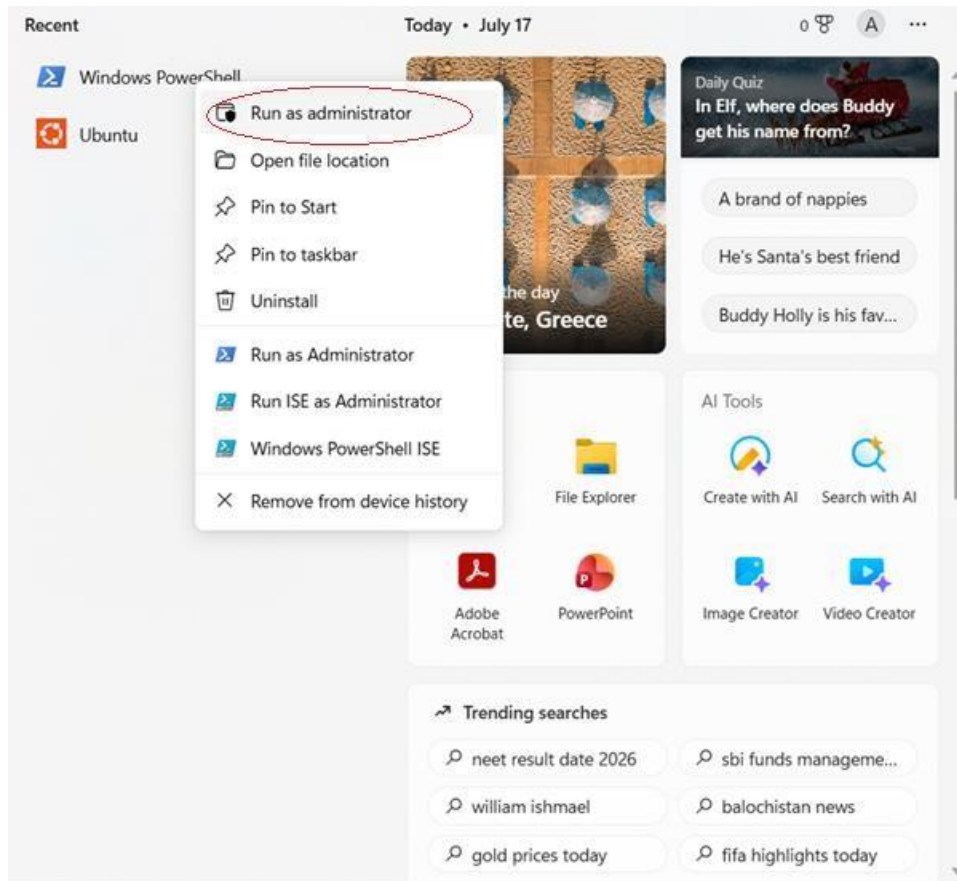


Step 1: Open PowerShell as Administrator

1. Press **Windows**.
2. Search for **PowerShell**.



3. Right-click **Windows PowerShell**.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\windows\system32> wsl --install
```

Step 3: Restart Your Computer

After installation, restart your PC when prompted.

Step 4: Complete Ubuntu Setup

After restarting:

1. Ubuntu opens automatically (or search for **Ubuntu** in the Start menu).
2. Wait for the installation to finish.
3. Create a Linux username.

Example:

Enter new UNIX username: anusha

Step 5: Create a Password

You'll then be prompted to create a password.

New password: ****

Retype new password:****

```
chakrika@blue:~$ sudo apt update
[sudo] password for chakrika:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [341 kB]
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [432 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [32.7 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [105 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [150 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [62.0 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [223 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translations-en [72.7 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [161 kB]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [331 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [63.0 kB]
Fetched 2292 kB in 6s (370 kB/s)
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Step 6: Update Ubuntu

```
chakrika@blue:~$ sudo apt install build-essential
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 87
```

Step 7: Install GCC (C Compiler)

Step 8: Verify the Installation

```
chakrika@blue:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Step 9: Create Your C Program

```
GNU nano 8.7.1 code.c *
#include <stdio.h>
int main() {
    printf("Hello World!\n");
    return 0;
}
```

Example: nano process.c

Step 10: Compile the Program

```
Hello World/chakrika@blue:~$ nano code.c
chakrika@blue:~$ gcc code.c -o code
chakrika@blue:~$ ./code
Hello World!
```

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:

```
OSSP/
|
|— .git/          # Git repository (created automatically)
|— .gitignore     # Files/folders Git should ignore
|— README.md      # Project description and instructions
|— LICENSE        # Project license (optional)
|— Makefile       # Build instructions
|
|— src/           # Source code
|   |— main.c
|   |— shell.c
|   |— parser.c
|   |— executor.c
|   └— builtin.c
|
|— include/       # Header files
|   |— shell.h
|   |— parser.h
|   |— executor.h
|   └— builtin.h
|
|— obj/           # Object files (.o) (generated)
```

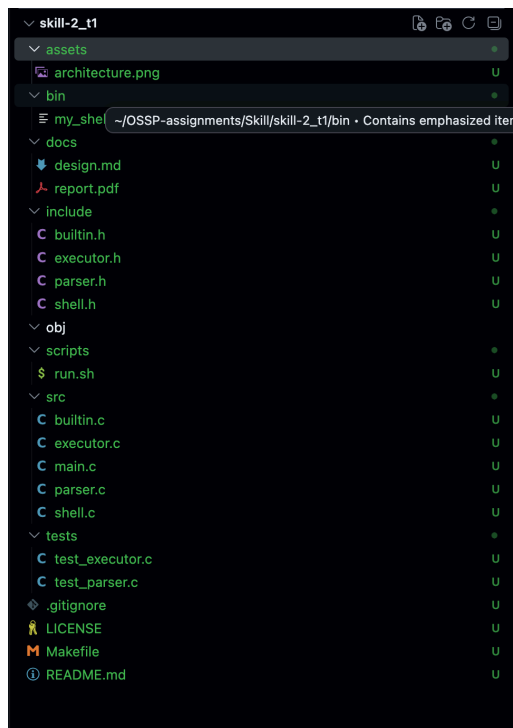
```

|
|— bin/          # Executable file (generated)
|   └─ my_shell
|
|
|— docs/         # Documentation
|   └─ design.md
|   └─ report.pdf
|
|
|— tests/        # Test programs
|   └─ test_parser.c
|   └─ test_executor.c
|
|
|— scripts/      # Helper scripts
|   └─ run.sh
|
|
└─ assets/       # Images/screenshots
    └─ architecture.png

```

Note: For creating this structure use mkdir and touch commands.

Structure:



2. To Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the `fork()` and `exec()` system calls

Output:

```
$ pstree -p $$
bash(2041)——pstree(2200)

$ strace -f -e trace=fork,vfork,clone,execve ls
execve("/bin/ls", ["ls"], 0x7ffe...) = 0
```

Task-2: Write a C program to create a new process and accepting the user commands using `fork()` and `exec()` system calls.

Example:

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
```

```

    // Child Process
    printf("This is the Child Process.\n");
    printf("Child PID = %d\n", getpid());
}
else {
    // Parent Process
    printf("This is the Parent Process.\n");
    printf("Parent PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);
}
return 0;
}

```

Task2 code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_LINE 256
#define MAX_ARGS 32

int main() {
    char line[MAX_LINE];
    char *args[MAX_ARGS];
    pid_t pid;

```

```

while (1) {
    printf("myshell> ");
    fflush(stdout);

    if (fgets(line, sizeof(line), stdin) == NULL)
        break;                // Ctrl+D (EOF)

    line[strcspn(line, "\n")] = '\0';    // strip newline

    if (strlen(line) == 0)
        continue;              // empty command
    if (strcmp(line, "exit") == 0)
        break;

    // Tokenize into argv[]
    int argc = 0;
    char *tok = strtok(line, " ");
    while (tok != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = tok;
        tok = strtok(NULL, " ");
    }
    args[argc] = NULL;

    pid = fork();              // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }
}

```

```

    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());

        execvp(args[0], args);          // Replace child image with the command
        // Only reached if execvp() fails
        printf("Command not found: %s\n", args[0]);
        exit(1);
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        int status;
        waitpid(pid, &status, 0);        // Wait for the child to finish
        if (WIFEXITED(status))
            printf("Child exited with status %d\n\n", WEXITSTATUS(status));
    }
}
return 0;
}

```

Output:

```
myshell> ls
This is the Parent Process.
Parent PID = 3050
Child PID = 3051
This is the Child Process.
Child PID = 3051
file1.txt  file2.txt  Makefile
Child exited with status 0
```

```
myshell> pwd
This is the Parent Process.
Parent PID = 3050
Child PID = 3052
This is the Child Process.
Child PID = 3052
/home/user/OSSP
Child exited with status 0
```